EDI Member 3 Project Work

==========

- **Topics to prepare**
  - Python
  - Federate Learning
  - LoRA adapter updates
  - NumPy Basics
  - Serialization
  - Compression
  - Client-Server communication (Flask/FastAPI or Sockets)
  - Secure Aggregation concepts
  - Differential Privacy

==========

- **Each Topic in Detail**
  - **Python**
    - Classes
    - Functions
    - File handling
    - OOP
    - Modules
    - Exception handling

==========

  - **NumPy Basics**
    - Arrays
    - Matrix operations
    - Saving/loading arrays
    - Basic indexing

==========

  - **Federated Learning**
    - Client-server architecture
    - Training rounds
    - FedAvg
    - Global model vs local model
    - Why data never leaves the client

==========

- **LoRA Adapter Updates**
  - What LoRA is
  - Adapter layers
  - Why only adapters are shared
  - How adapter weights are stored

==========

- **Serialization….(pickle)**
  - pickle.dumps()
  - pickle.loads()
  - Converting model updates into bytes

==========

- **Compression….(gzip)**
  - Compressing serialized updates
  - Decompressing them
  - Why compression reduces communication cost

==========

- **Client Server Communication**
  - HTTP basics
  - REST APIs
  - Flask/FastAPI
  - Sending files/bytes
  - Request and response cycle

==========

- **Secure Aggregation**
  - Purpose
  - How encrypted updates can be aggregated
  - Prototype implementation (as described in the PDF)

==========

- **Differential Privacy**
  - Noise addition
  - Privacy budget (basic idea)
  - Why it protects client information
  - Prototype-level implementation

==========

- ## Importance of Each Topic
  - ### Python and NumPy

    The whole project will be made up in python file like aggregation.py, communication.py , compression.py, and serializer.py will play a major role in the project.

    Apart from this numpy is equally important because, A neural network is basically millions of numbers, the model store numbers instead of storing strings like cat,dog,bird it stores 0.45,0.23,-0.91 respectively which are called weights.

    LoRA adapters are also collections of weights.Those weights are stored as an array using numPy.

  - ### Federated Learning

    This is most important because the entire project is dependent on federated Learning.
    Actually in the traditional model the privacy of the users gets lost.
    In this process multiple clients train their model locally and then combine them for the final model so no loss of privacy.
    When Member 2 produces the LoRA Adapters, member 3's task is to process those LoRA Adapters to prevent privacy.

  - ### LoRA Adapters Updates

    It often happens that the large language model may have 7 Billion Parameters so sending them in one round would be huge.
    Instead of that LoRA helps to train the tiny model for example it may update 5 million parameters which is much smaller and faster than the existing model.

- ## Serialization

  Computers cannot send the python objects (dictionary,list,NumPy array,Tensor) directly so they need to be converted into bytes.
  Serialization helps to convert the python object into bytes.
  Python Objects → Bytes → Network
  To convert into bytes in serialization we use `pickle.dump()`
  Adapter → pickle.dump() → Bytes

- ## Compression

  Compression helps to reduce the size of the adapters which decrease the load on the system and use minimum resources.
  Before Adapter Size (120 MB) → gzip → After Adapter Size (32 MB)
  Serialized Bytes → gzip → Compressed Bytes → Send

- ## Client Server Communication

  Clients and the server are separate programs (or even separate computers).
  They cannot directly access each other's memory.
  They communicate over a network using requests and responses.
  Client-Server Communication is the mechanism through which clients send serialized and compressed LoRA updates to the central server, and the server receives them for aggregation.

  LoRA Adapter→Serialize→Compress→Send to Server→Server Receives

- **Secure Aggregation**

  Secure Aggregation is a privacy technique where the server combines updates from all clients without seeing any individual client's update.
  Instead of sending raw updates directly,your privacy module protects them before they reach the server.

  The server only receives the combined update, not each client's individual update. The PDF states that, for this prototype, secure aggregation can be simulated rather than implemented with industrial-grade cryptography.

  Secure Aggregation is a privacy mechanism that allows the server to combine model updates from multiple clients without seeing any client's individual update.

- **Differential Privacy**

  Differential Privacy is a technique that adds small random noise to a client's model update so that private training data cannot be easily inferred, while still allowing the model to learn.